Junior AI Engineer — One Day Take-Home Test

Time: 1 day maximum **Submission:** GitHub repository link

Scenario

You are building an AI agent for **StayEase**, a short-term accommodation rental platform in Bangladesh. Guests message the agent to search for available properties and make bookings.

The agent should handle these three things only:

1. **Search** — Guest provides location, dates, and number of guests. Agent returns available properties.
2. **Details** — Guest asks about a specific property. Agent returns listing information.
3. **Book** — Guest confirms they want to book. Agent creates the booking.

If the agent cannot handle anything outside these three, it escalates to a human.

What to Submit

---

1. Architecture Document (`README.md`)

1.1 System Overview Write a short paragraph explaining what the system does and include one **Mermaid diagram** showing how these components connect:

- FastAPI backend
- LangGraph agent
- PostgreSQL database
- Groq / OpenRouter LLM

1.2 Conversation Flow Walk through one complete example step by step — from the moment a guest says *"I need a room in Cox's Bazar for 2 nights for 2 guests"* all the way to the agent returning available properties with prices.

1.3 LangGraph State Design Define the state object with field names and types. Explain in one line why each field is needed.

1.4 Node Design List each node in your graph. For each node write:

- What it does (one sentence)
- What it updates in state
- What node comes next

Keep it to **4–5 nodes maximum.**

1.5 Tool Definitions Define these **3 tools** with input parameters, output format, and when the agent uses each:

- `search_available_properties`
- `get_listing_details`
- `create_booking`

1.6 Database Schema Design 3 tables only:

- `listings`
- `bookings`
- `conversations`

Provide column names and data types for each.

---

2. LangGraph Agent Skeleton (Python)

Write skeleton code organized in this structure:

```
agent/
  state.py      # State definition
  nodes.py      # Node functions
  tools.py      # Tool definitions
  graph.py      # Graph construction
```

Requirements:

- State class using `TypedDict`
- At least **3 node functions** with type hints and a short docstring each
- Full graph definition with nodes, edges, and conditional routing
- At least **2 tools** using `@tool` decorator with Pydantic input schemas

Code does not need to run end-to-end but must be clean, typed, and logically correct.

---

3. API Contract (`api.md`)

Document these 2 endpoints:

`POST /api/chat/{conversation_id}/message` — Send a guest message

`GET /api/chat/{conversation_id}/history` — Get conversation history

For each provide:

- Request and response schema
- One realistic example using Bangladeshi locations and BDT pricing
- Possible error responses

Evaluation Criteria

Area	Weight	What we look for
Architecture document	40%	Is the design clear and logical? Could a developer build from it?
LangGraph understanding	30%	Correct use of state, nodes, edges, and tools
Code quality	20%	Clean, typed, well structured
API design	10%	Realistic examples, correct HTTP conventions

Rules

- Python, LangGraph, LangChain, FastAPI only
- Mermaid diagram in README (renders on GitHub)
- Type hints required throughout
- Public GitHub repo with **multiple clean commits** — do not push everything at once

---

Submission

Push to GitHub and share your repository link.